EDS Laboratory

Practical No.1

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: p=mv

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
m=float(input())

v=float(input())

p=m*v
```

print("%.2fkgm/s"%p)

Output:

Test case 1 8 ms Debug

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

Test case 2 6 ms Debug

Expected output	Actual output
2.3	2.3
4.8	4.8
11.04kgm/s	11.04kgm/s

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

1 ≤ n ≤ 999

Input Format:

The input consists of a single integer .

Output Format:

If n is a single-digit number, print its square.

If n is a two-digit number, print its square root (rounded to two decimal places).

If n is a three-digit number, print its cube root (rounded to two decimal places). Else print "Invalid".

Code:

```
a=int(input())
if (0<a<10):
    print(a**2)
elif (9<a<100):
    print(f"{a**(1/2):.2f}")
elif(99<a<1000):
    print(f"{a**(1/3):.2f}")
else:
   print("Invalid")
```

Output:

Test case 1 13 ms Debug

Expected output	Actual output
9	9
81	81

Test case 2 10 ms Debug

Expected output	Actual output
166	166
5.50	5.50

Test case 3 10 ms Debug

Expected output	Actual output
24	24
4.90	4.90

1.1.3. Age and Salary Calculation

Write a Python program that reads the birth date and salary of employees.

Input Format:

The input consists of:

A string representing the birth date of the employee in the format .

A floating-point number representing the salary of the employee in rupees.

Output Format:
The output should include:
The age of the employee.
The salary of the employee in dollars.
Note:
1INR=0.012USD

Code:

```
def calculate_age(birthdate):
birthdate = list(map(int,birthdate.split("-")))
return 2024 - birthdate[2]
 def convert_salary_to_dollars(salary_in_rupees):
return salary_in_rupees * 0.012
 birthdate = input()
salary_in_rupees = float(input())
age = calculate_age(birthdate)
salary_in_dollars = convert_salary_to_dollars(salary_in_rupees)
print(f"Age: {age}")
print(f"Salary in dollars: {salary_in_dollars:.2f}")
```

Output:

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number.

Code:

```
num= int(input())
reversed_num=int(str(num)[::-1])
print(reversed_num)
```

Output:

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

The character "x" is used in the output to represent multiplication.

Refer to the visible test-cases for more clarity.

Code:

```
num  = int(input())
 for i in range(1 ,11):
print(f"{num} x {i} = {num * i}")
```

Output:

Test case 1 12 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Test case 2 7 ms Debug

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Code:

```
n=int(input())
marks=list(map(int,input().split()))
if len(marks)!=n:
print("Invalide")
else:
if any(mark<40 for mark in marks):
print("Fail")
else:
total=sum(marks)
percentage=total/n
print("Aggregate Percentage:",f"{percentage:.2f}")
```

```
if percentage>75:
print("Grade: Distinction")
elif 60<=percentage<75:
print("Grade: First Division")
elif 50<=percentage<60:
print("Grade: Second Division")
elif 40<=percentage<50:
print("Grade: Third Division")
```

Output:

Test case 1 12 ms Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2 11 ms Debug

Expected output	Actual output
5	5
56 78 97 86 93	56 78 97 86 93
Aggregate Percentage: 82.00	Aggregate Percentage: 82.00
Grade: Distinction	Grade: Distinction

Test case 3 9 ms Debug

Expected output	Actual output
5	5
99 85 43 97 34	99 85 43 97 34
Fail	Fail

Test case 4 7 ms Debug

Expected output	Actual output
3	3
55 54 53	55 54 53
Aggregate Percentage: 54.00	Aggregate Percentage: 54.00
Grade: Second Division	Grade: Second Division

1.2.2. Fibonacci series using Recursive Function

Write a Python program to find the Fibonacci series of a given number of terms using recursive function calls.

Expected Output-1:

Enter terms for Fibonacci series: 5

0 1 1 2 3

Expected Output-2:

Enter terms for Fibonacci series: 9

0 1 1 2 3 5 8 13 21

Instructions:

Your input and output must follow the input and output layout mentioned in the visible sample test case.

Hidden test cases will only pass when users' input and output match the expected input and output.

Code:

```
def fibonacci(n):
    if n==1:
return 0
elif n==2:
return 1
else:
```

```
return fibonacci( n-1 ) +fibonacci(n-2)
 n = int(input())
for i in range(1, n + 1):
print(fibonacci(i), end=" ")
```

Output:

Test case 1 8 ms Debug

Expected output	Actual output
Enter terms for Fibonacci series: 5	Enter terms for Fibonacci series: 5
0 1 1 2 3	0 1 1 2 3

Test case 2 5 ms Debug

Expected output	Actual output
Enter terms for Fibonacci series: 9	Enter terms for Fibonacci series: 9
0 1 1 2 3 5 8 13 21	0 1 1 2 3 5 8 13 21

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

*	
* *	
* * *	
* * * *	

Code:

```
a=int (input())
 i= 1
while i < a+1:
    print("* "*i)
    i+=1
```

Output:

Test case 1 10 ms Debug

Expected output	Actual output
5	5
*.	*.
..	*.*.
..*.	*.*.*.
..*.*.	*.*.*.*.
..*.*.*.	*.*.*.*.*.

Test case 2 4 ms Debug

Expected output	Actual output
4	4
*.	*.
..	*.*.
..*.	*.*.*.
..*.*.	*.*.*.*.

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

Test case 1

5	
1·⏎	
1·2·⏎	
1·2·3·⏎	
1·2·3·4·⏎	
1·2·3·4·5·⏎	

Code:

```
a=int (input())
i=1
for i in range (1,1+a):
for j in range (1,i+1):
```

```
print(j,end= " ")
print("")
```

Output:

Test case 1 7 ms Debug

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Test case 2 8 ms Debug

Expected output	Actual output
8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8